



ANGELES UNIVERSITY FOUNDATION

Mac Arthur Highway, Angeles City, Philippines, 2009

BSIT 3A - A.Y. 2026-2027



The Builder Design Pattern in Modern PHP: Taming Complex Object Construction Without Telescoping Constructors

A Research Article Submitted for
IPT10 — Integrative Programming and Technologies

Submitted by
Bea Louise Uson

Submitted to
Mr. Romack L. Natividad

September 26, 2026



Introduction / Project Overview

In modern OOP systems, there are times when an object needs to be constructed with several options and interdependent parts. The most common but poor solution is the *telescoping constructor*, where a single constructor has too many parameters and most of them are optional. Because of this, callers must remember a long list of parameters just to create one object, making it confusing for the developers. For example, in a food-ordering application, we have a 'ComboMeal' class that may need a main and side dish, drink, size, and several other add-ons that are optional. Without a dedicated construction mechanism, developers might overload the constructor with a lot of parameters and scatter conditional logic for different meal types, making the code harder to maintain because every new combo type may require changing the existing code.

It was stated by Gamma et al. (1994), the Builder pattern is used to separate the process of creating a complex object from the object itself. This allows the same step-by-step construction process to create different representations of the finished object.

The purpose of the Builder design pattern is to help solve complex object creation by giving the construction process its own class hierarchy. It separates the construction of the object from the object itself and from the code that will use the finished object, making the code more organized and easier to modify when new options or variations are added.

Literature & Theory

Five sources ground this article in both the Builder pattern's original formulation and its contemporary treatment within the PHP ecosystem. Gamma, Helm, Johnson, and Vlissides (1994), the "Gang of Four" (GoF), introduced Builder as one of twenty-three cataloged design patterns, describing its structure, participants (Builder, ConcreteBuilder, Director, Product), and applicability on pp. 97–106. This is the canonical definition against which later, language-specific treatments are measured, and it is the source of the intent statement quoted on Page 1.

Freeman and Robson (2021), in the second edition of Head First Design Patterns, give Builder a brief overview in the book's "Leftover Patterns" appendix rather than a full chapter, since they judged it less commonly used day-to-day than patterns like Observer or Strategy. Even so, their treatment usefully distinguishes Builder from the related Abstract Factory pattern: Abstract Factory returns a finished product in one call, while Builder assembles it across several calls before the client requests the result.



Zandstra (2016), writing specifically for PHP developers, demonstrates Builder using PHP's own object model and connects it to fluent method chaining (returning \$this from each setter), a technique common in PHP query and form builders.

For empirical grounding, Ampatzoglou, Chatzigeorgiou, Charalampidou, and Avgeriou (2015) found that GoF patterns, Builder included, improve the structural stability of evolving object-oriented systems (IEEE Transactions on Software Engineering, vol. 41, no. 8, pp. 781–802).

Structure and Diagram

The Builder pattern has four participants. The Builder is an interface declaring one method per construction step plus a method that returns the finished product. A ConcreteBuilder (here, ValueComboBuilder and DeluxeComboBuilder) implements that interface and keeps track of the product under construction. The Director calls the builder's steps in a fixed sequence, without knowing which concrete builder or product variant it is assembling. The Product (ComboMeal) is the complex object being built.

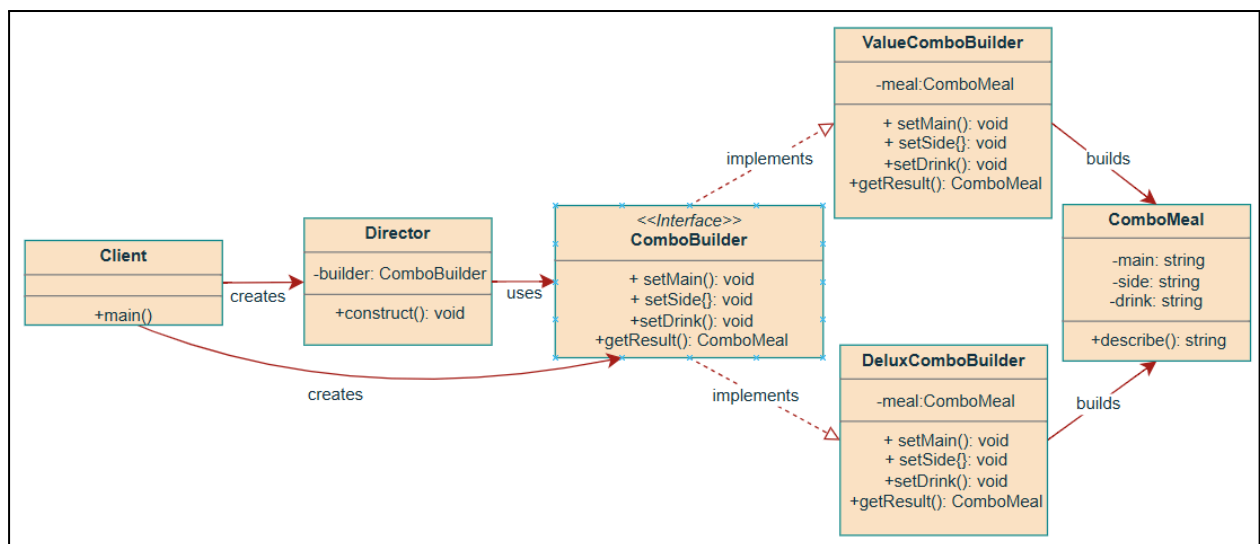


Figure 1. UML class diagram of the Builder pattern applied to a combo-meal ordering system.

Source: diagram by the author, adapted from the canonical Builder structure in Gamma et al. (1994, p. 97).

Figure 2 traces the runtime collaboration between the participants: the client selects a concrete builder, hands it to the director, the director drives the three construction steps, and the client finally retrieves the finished product from the builder.

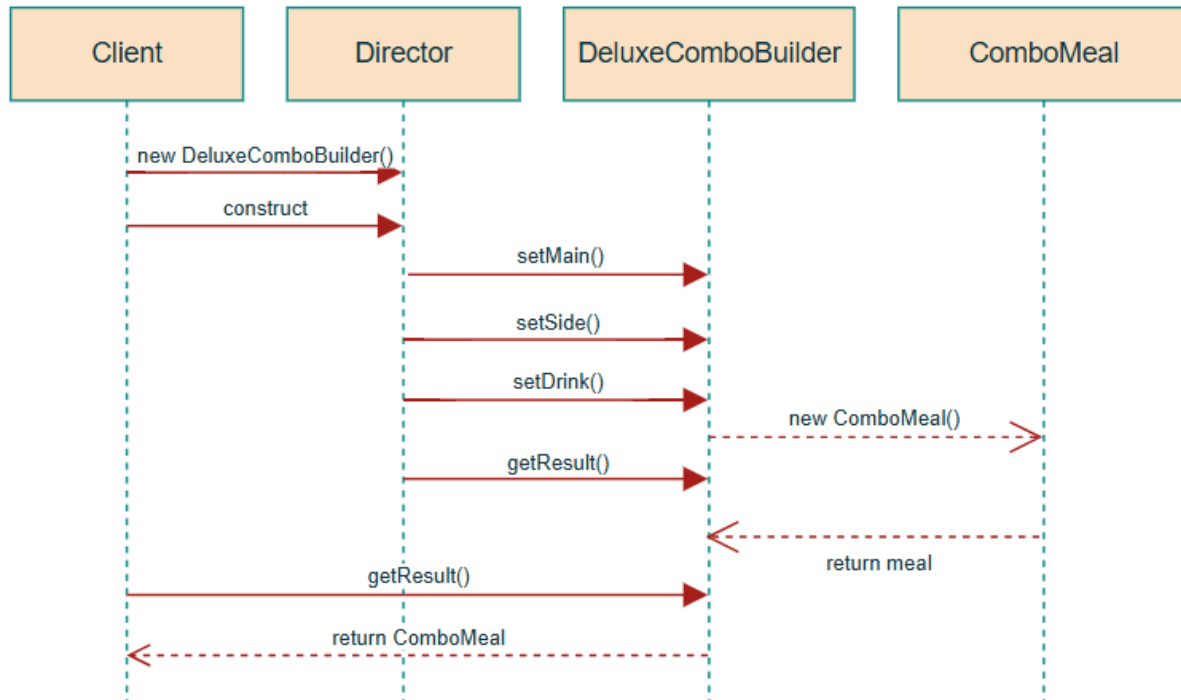


Figure 2. Sequence diagram — building a Deluxe combo meal.

Source: original diagram created by the author.

PHP Implementation

The listing below is a combo meal builder, original PHP 8.1+ implementation written for a food-ordering module. It uses typed properties, an interface for the Builder contract, and readonly value objects for the finished product, and it passes php -l with no syntax errors.

ComboBuilder:

```
1 <?php
2
3 declare(strict_types=1);
4
5 namespace App;
6
7 /**
8  * Builder
9  */
10 interface ComboBuilder
11 {
12     public function setMain(string $main): static;
13     public function setSide(string $side): static;
14     public function setDrink(string $drink): static;
15     public function getResult(): ComboMeal;
16 }
```

ComboDirector:

```
1 <?php
2
3 declare(strict_types=1);
4
5 namespace App;
6
7 /**
8  * Director
9  */
10 final class ComboDirector
11 {
12     public function construct(ComboBuilder $builder, string $main, string $side, string $drink): ComboMeal
13     {
14         $builder->setMain($main)
15             ->setSide($side)
16             ->setDrink($drink);
17     }
18 }
```



ComboMeal:

```
<?php
declare(strict_types=1);

namespace App;

/**
 * Product.
 */
/**
 * references
 */
final class ComboMeal
{
    /**
     * @param array<int, string> $addOns
     */
    /**
     * references
     */
    public function __construct(
        public readonly string $main,
        public readonly string $side,
        public readonly string $drink,
        public readonly array $addOns = []
    ) {
    }

    /**
     * references
     */
    public function describe(): string
    {
        $extras = $this->addOns === [] ? 'none' : implode(', ', $this->addOns);

        return sprintf(
            '%s + %s + %s (add-ons: %s)',
            $this->main,
            $this->side,
            $this->drink,
            $extras
        );
    }
}
```

DeluxeComboBuilder:

```
<?php
declare(strict_types=1);

namespace App;

/**
 * ConcreteBuilder.
 */
/**
 * references
 */
final class DeluxeComboBuilder implements ComboBuilder
{
    /**
     * references
     */
    private string $main = 'Grilled Chicken';
    /**
     * references
     */
    private string $side = 'Mashed Potato';
    /**
     * references
     */
    private string $drink = 'Fruit Shake';
    /**
     * references
     */
    private array $addOns = ['Extra Gravy', 'Upsized Drink'];

    /**
     * references
     */
    public function setMain(string $main): static
    {
        $this->main = $main;
        return $this;
    }

    /**
     * references
     */
    public function setSide(string $side): static
    {
        $this->side = $side;
        return $this;
    }

    /**
     * references
     */
    public function setDrink(string $drink): static
    {
        $this->drink = $drink;
        return $this;
    }

    /**
     * references
     */
    public function getResult(): ComboMeal
    {
        return new ComboMeal($this->main, $this->side, $this->drink, $this->addOns);
    }
}
```

ValueComboBuilder:

```
<?php
declare(strict_types=1);

namespace App;

/**
 * ConcreteBuilder.
 */
/**
 * references
 */
final class ValueComboBuilder implements ComboBuilder
{
    /**
     * references
     */
    private string $main = 'Fried Chicken';
    /**
     * references
     */
    private string $side = 'Rice';
    /**
     * references
     */
    private string $drink = 'Iced Tea';

    /**
     * references
     */
    public function setMain(string $main): static
    {
        $this->main = $main;
        return $this;
    }

    /**
     * references
     */
    public function setSide(string $side): static
    {
        $this->side = $side;
        return $this;
    }

    /**
     * references
     */
    public function setDrink(string $drink): static
    {
        $this->drink = $drink;
        return $this;
    }

    /**
     * references
     */
    public function getResult(): ComboMeal
    {
        return new ComboMeal($this->main, $this->side, $this->drink);
    }
}
```

Syntax error detection:

```
C:\Users\lbea6\Downloads\combo-meal-builder>php -l src/ComboMeal.php
No syntax errors detected in src/ComboMeal.php

C:\Users\lbea6\Downloads\combo-meal-builder>php -l src/ComboBuilder.php
No syntax errors detected in src/ComboBuilder.php

C:\Users\lbea6\Downloads\combo-meal-builder>php -l src/ValueComboBuilder.php
No syntax errors detected in src/ValueComboBuilder.php

C:\Users\lbea6\Downloads\combo-meal-builder>php -l src/DeluxeComboBuilder.php
No syntax errors detected in src/DeluxeComboBuilder.php

C:\Users\lbea6\Downloads\combo-meal-builder>php -l src/ComboDirector.php
No syntax errors detected in src/ComboDirector.php

C:\Users\lbea6\Downloads\combo-meal-builder>php -l public/index.php
No syntax errors detected in public/index.php

C:\Users\lbea6\Downloads\combo-meal-builder>cd C:\Users\lbea6\IPT RESEARCH MIDTERM
```

Execution:

```
C:\Users\lbea6\Downloads\combo-meal-builder>php public/index.php

=====
Builder Design Pattern in PHP - Combo Meal Demo
=====

1. Deluxe combo (built via Director):
   Grilled Chicken + Mashed Potato + Fruit Shake (add-ons: Extra Gravy, Upsized Drink)

2. Value combo (built via Director):
   Fried Chicken + Rice + Iced Tea (add-ons: none)

3. Custom combo (built via fluent chaining, no Director):
   Beef Tapa + Garlic Rice + Calamansi Juice (add-ons: Extra Gravy, Upsized Drink)

Execution completed successfully.
```

Github repository: <https://github.com/bea-blue/design-patterns-laboratory>

Application and Reflection

Mini Case Study

Laravel's Eloquent QueryBuilder is a production example of the same idea applied to SQL generation: calling `->where()`, `->orderBy()`, and `->limit()` incrementally assembles a query object step by step, and the final SQL string (the Product) is only produced when `->get()` is called. A capstone or thesis project built on Laravel or Symfony, such as an AUF student information add-on, or a food-ordering module resembling the example on Page 4 can use the same approach any time a form, a report, or an API payload has several optional fields that would otherwise bloat a single constructor.

When to Use	When Not to Use
-------------	-----------------



An object has many optional or interdependent parts (a form, a query, a meal, an HTTP request).	The object has two or three simple, required fields, a plain constructor is clearer.
The same construction steps must produce several different representations of the result.	There is only ever one representation to build Factory Method is simpler.
You want construction logic isolated from business logic for easier testing.	Adding a Builder/Director hierarchy would outweigh the complexity it removes (over-engineering).

Reflection

After working through this pattern I noticed that without builder pattern, ComboMeal would need one constructor call with up to six arguments in a fixed order, and every new combo type Value, Deluxe, or a custom one would mean either passing unused parameters or writing a new constructor overload. Splitting construction into `setMain()`, `setSide()`, and `setDrink()` instead meant each piece could be set independently, and the Director could reuse the exact same three-step sequence regardless of which concrete builder it was handed.

The part that took the most effort to actually understand was the sequence diagram. When I first drew it, I had the return meal arrow pointing the wrong way, it was pointing toward ComboMeal instead of back from it. Understanding why new ComboMeal() has to hand a reference back to the builder, otherwise the object would just vanish after being created was the moment the pattern actually clicked, more than reading the GoF definition did.

What felt familiar, on the other hand, was the fluent chaining (`->setMain()->setSide()->setDrink()`). I'd already seen and half-used that style without knowing it had a formal name or fit into a named design pattern.

Beyond the code, I also picked up a practical lesson outside the pattern itself: setting up the project with Composer's PSR-4 autoloading, running `php -l` on every file, and pushing it to GitHub gave me a small taste of how a real PHP project is organized and verified before it's considered "done", not just written once and left alone.



References

- Ampatzoglou, A., Chatzigeorgiou, A., Charalampidou, S., & Avgeriou, P. (2015). The effect of GoF design patterns on stability: A case study. *IEEE Transactions on Software Engineering*, 41(8), 781–802. <https://doi.org/10.1109/TSE.2015.2414917>
- Freeman, E., & Robson, E. (2021). *Head first design patterns: Building extensible and maintainable object-oriented software* (2nd ed.). O'Reilly Media.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.
- Refactoring.Guru. (n.d.). Builder. Retrieved 2026, from <https://refactoring.guru/design-patterns/builder>
- Zandstra, M. (2016). *PHP objects, patterns, and practice* (5th ed.). Apress. <https://doi.org/10.1007/978-1-4842-1996-6>